

LOOPS.md: Field Notes on Agents That Run for Days

A Short List of Rules for Letting the Model Drive

Andrej Karpathy
Independent Researcher

Abstract. This file exists because most agent systems die not from a weak model but from a weak harness. The model can write code; the model can review code; the model can verify its own output against a rubric it agreed to ten minutes ago. What it cannot do, on its own, is decide when to stop, when to restart, and where to write the result. That is the work of the loop. The pattern in this note treats the loop as a first-class object: roles are separated, state lives on disk, contracts are negotiated between agents before the first line of code is written, and the harness is read like a stack trace whenever something goes wrong. Short loops, simple state, clean contracts. Everything else is decoration.

Index Terms. agentic loops, Claude Code, harness design, generator-evaluator pattern, sprint planning, file-system state, contract negotiation, trace reading, deletable scaffolding.

I. WRITE THE LOOP, NOT THE PROMPT

A prompt is a thing you type once and forget. A loop is a thing that runs while you sleep. The unit of leverage stopped being the prompt the moment models became good enough to follow a procedure without supervision; what matters now is the procedure. If you find yourself iterating on a single message at three in the morning, you are still in the prompting era. Close the tab. Write the loop. The loop is short: gather, reason, act, verify, repeat. Everything in this document is a footnote on those five verbs.

II. SEPARATE THE ROLES

Three roles, three context windows, three system prompts. A planner that turns a vague human sentence into a sprint spec and never touches code. A generator that writes everything and is forbidden from grading its own work. An evaluator that reads diffs, launches playwright, plays the app, and is told from the first message that the code is broken and its job is to prove it. Mixing the roles is the most common failure I see; the model becomes sycophantic the moment it grades itself, and the loop quietly converges on slop.

III. NEGOTIATE THE CONTRACT FIRST

Before the generator writes a single line, it proposes what done looks like and the evaluator pushes back. The two argue via markdown files on disk until they agree on a checklist of testable assertions. Twenty-seven criteria is a reasonable size for a small app; ten is usually too few and the evaluator rubber-stamps. The original spec from the planner is the boundary, but the contract is what gets graded. This is the single change that moved my own runs from broken demos to working products.

IV. WRITE TO DISK, NOT TO CONTEXT

Context windows lie. They compact, they rot, they hide what you said an hour ago behind a summary you did not write. A file on disk does not lie. Keep `feature_list.json`, `progress.md`, `contract.md`, and an append-only `log.md` with `## [YYYY-MM-DD] op | title`

entries. The model should be able to crash, lose its session, and pick up where it left off by reading three files. If you cannot describe your state in three files, your state is too complicated.

V. LET THE LOOP RESTART

Counter-intuitively, the best behavior I see from current frontier models is the willingness to throw everything away and start over when a run goes sideways. Older models patched and patched until the codebase resembled archaeology; newer ones, given a clean evaluator and a contract on disk, will delete the project at iteration nine and ship a working version at iteration eleven. Do not interrupt this. The restart is the loop working correctly. Insert a human only when the contract itself is wrong, not when the build is.

VI. SCORE THE SUBJECTIVE

Taste is gradable if you write it down. Four axes, weighted: design, originality, craft, functionality. Calibrate on three reference sites the evaluator is told are good and three it is told are slop. The output is a number between zero and one and a paragraph explaining the gap. The model will not invent taste; it will only converge toward the taste you described. The whole game is writing the rubric carefully enough that converging toward it is what you actually wanted.

VII. READ THE TRACES

Every debugging insight I have about agent loops came from reading the raw transcript, not from running another experiment. Pipe the agent's output into a file, grep for the moment its judgment diverged from yours, edit the prompt for that exact moment, run again. This is the same muscle as reading a stack trace; the difference is that the trace is written in English and most of it is the model talking to itself. Skip this step and you are tuning by vibe.

VIII. DELETE THE HARNESS

The harness exists to compensate for the model. As the model improves, half of what you wrote last quarter becomes overhead. Context resetting between sessions was load-bearing for one model generation and dead weight for the next; sprint decomposition was the only thing keeping a four-hour build coherent and is now a constraint on a model that holds two hours in one head. Re-read your harness against each new release and delete anything the model now does for free. The harness that grows monotonically is a harness you have stopped reading.

IX. THE BOTTLENECK ALWAYS MOVES

When coding stops being the bottleneck, planning becomes the bottleneck. When planning is solved, verification becomes the bottleneck. When verification is automated, taste becomes the bottleneck. You do not finish; you find the next thing to fix. The whole point of the loop is to make the next bottleneck visible. If everything is going smoothly, you are not looking carefully enough. Find the new bottleneck, fix it, ship a smaller harness, repeat.